분류	수식	변수 설명
[Ch0] 전력	$P \propto C \cdot V^2 \cdot F$	C: 커패시턴스, V: 전압, F: 주파수 (Dennard Scaling)
[Ch1] SOP	F = Σ(minterms where Y=1)	minterm의 OR, 정규 곱의 합
[Ch1] POS	F = Π(maxterms where Y=0)	maxterm의 AND, 정규 합의 곱
[Ch1] Combining	$P \cdot A + P \cdot \bar{A} = P$	변수 A 제거 (K-Map의 핵심 원리)
[Ch1] Covering	$B \cdot (B+C) = B$	흡수법칙
[Ch1] Consensus	$BC + \bar{B}D + CD = BC + \bar{B}D$	중복 항 제거
[Ch1] De Morgan	$\overline{(B_0 \cdot B_1 \ldots)} = \bar{B}_0 + \bar{B}_1 \ldots$	AND↔OR 반전 교환
[Ch1] De Morgan (dual)	$\overline{(B_0 + B_1 \ldots)} = \bar{B}_0 \cdot \bar{B}_1 \ldots$	
[Ch1] Mux	$Y = D_0 \cdot \bar{S} + D_1 \cdot S$	2:1 Mux 수식
[Ch1] FSM next state (예)	$S'_1 = S_1 \oplus S_0$	Traffic Light Controller
[Ch1] FSM next state (예)	$S'_0 = \bar{S}_1 \cdot \bar{S}_0 \cdot \bar{T}_A + S_1 \cdot \bar{S}_0 \cdot \bar{T}_B$	
[Ch3] 4-bit Adder	{c_out, sum} = x + y + c_in	concatenation으로 5비트 결과 분리
[Ch3] Adder/Subtractor	t = y ^ {4{c_in}}; {c_out,sum} = x + t + c_in	c_in=0→덧셈, c_in=1→뺄셈(2의 보수)
[Ch3] Even Parity	ep = ^x	XOR reduction, 모든 비트 XOR
[Ch3] All-Zero	zero = ~(|x)	OR reduction 후 NOT
[Ch3] All-One	one = &x	AND reduction
[Ch4] D FF	always @(posedge CLK) Q <= D;	nonblocking, rising edge에서 D→Q
[Ch4] D Latch	always @(CLK,D) if(CLK) Q<=D;	CLK=1이면 transparent, 아니면 latch
[Ch4] Counter	always @(negedge clk or posedge clr)	비동기 리셋 + negedge 카운트
[Ch5] Hazard 제거	$f = xy + \bar{x}z + yz$	중복 항(yz) 추가로 static-1 hazard 제거
[Ch5] Gate delay	#(t_rise, t_fall, t_off)	rise:→1, fall:→0, turn-off:→z
[Ch5] Missing	t_off = min(t_rise,	2개만 지정 시

turn-off	t_fall)	
[Ch6] Function	function [3:0] func(input [7:0] d);	단일 반환, 조합논리 전용, 타이밍 불가
[Ch6] Task	task name(input d, output count);	여러 반환, 타이밍 가능

비교 항목	A	B
Combinational vs Sequential	출력 = f(현재 입력만), 메모리 없음	출력 = f(현재+과거 입력), 메모리 있음
Latch vs Flip-Flop	Level-sensitive (CLK=1일 때 transparent)	Edge-triggered (rising edge에서만 값 갱신)
SR Latch vs D Latch	S,R 두 입력 (S=R=1 금지상태 존재)	D,CLK 두 입력 (금지상태 없음, what/when 분리)
Moore vs Mealy FSM	출력 = f(state만), 안정적, 한 클럭 늦음	출력 = f(state+input), 빠른 반응, glitch 주의
SOP vs POS	minterm(Y=1)의 OR	maxterm(Y=0)의 AND
Minterm vs Maxterm	모든 변수의 AND (min satisfiability)	모든 변수의 OR (max satisfiability)
wire vs reg [Ch2]	continuous assign(assign)의 LHS, 값 저장 불가	procedural block(always)의 LHS, 반드시 물리 레지스터는 아님
Structural vs Dataflow vs Behavioral [Ch2]	게이트/모듈 인스턴스화	assign+연산자, 조합논리
Continuous vs Procedural Assignment [Ch2]	assign으로 wire 구동, 항상 활성	always 블록 내에서 reg에 할당
Synthesizable vs Non-synthesizable [Ch2]	module, always, assign, wire, reg	initial, #delay, $display, $monitor, $finish
Moore's Law vs Dennard Scaling [Ch0]	트랜지스터 수 ~2배/18-24개월 (둔화중)	전력밀도 일정 (이미 종료)
Inertial vs Transport delay [Ch3]	continuous assign 기본, 짧은 펄스 무시	net delay 기본, 모든 변화 전파
Logical vs Case equality [Ch3]	== : x/z 있으면 결과 x	=== : x/z까지 정확 비교 (합성 불가)
Logical vs Arithmetic shift [Ch3]	<<,>> : 빈 자리 항상 0	<<<,>>> : 산술 우시프트만 부호비트 패딩
Logical vs Bitwise operator [Ch3]	!,&&,|| : 전체→1비트 true/false	~,&,|,^ : 비트별 연산
Reduction vs Bitwise	단항, 벡터→스칼라	이항, 비트별 연산

[Ch3]	(&x, |x, ^x)	(a&b, a|b)
signed vs unsigned 비교 [Ch3]	둘 다 signed→sign extension+signed 비교	한쪽이라도 unsigned→zero-pad+unsigned 비교
Blocking vs Nonblocking [Ch4]	= : 순서대로, 즉시 할당, 조합논리용	<= : RHS 동시 평가, 블록 끝 할당, 순차논리용
initial vs always [Ch4]	시뮬레이션 중 1회만, 합성불가(ASIC)	이벤트마다 반복, 합성 가능
Latch vs Flip-Flop 합성 [Ch4]	always@(CLK,D) + if(CLK) → latch	always@(posedge CLK) → flip-flop
Inter vs Intra-assignment delay [Ch4]	#10 a=b; 문장 전체 지연	a=#10 b; RHS 즉시평가, 할당만 지연
case vs casex vs casez [Ch4]	0,1,x,z 정확 비교	casex: x,z don't care / casez: z don't care
wire vs wand vs wor [Ch5]	wire: 충돌→x	wand: 하나가 0→0 / wor: 하나가 1→1
Static vs Dynamic hazard [Ch5]	출력 일정해야 하는데 잠깐 반대 값	한 번 변해야 하는데 3+회 변함 (경로 3+개)
Gate(inertial) vs Wire(transport) delay [Ch5]	짧은 펄스 무시, 게이트 모델	모든 변화 전파, 와이어 모델
Function vs Task [Ch6]	input만, 단일 반환, 타이밍 불가, 조합 전용	in/out/inout, 여러 반환, 타이밍 가능, 범용
Static vs Automatic (Verilog) [Ch6]	기본값, 이전 호출 값 유지, 동시실행 위험	호출마다 새 할당, 재귀/동시 안전
Combinational vs Sequential UDP [Ch6]	출력=f(현재입력), reg 아님	출력=reg, 상태 테이블, initial 가능

용어	정의
always block	sensitivity list의 이벤트 발생 시 실행되는 procedural block [Ch2]
assign	continuous assignment 키워드, wire에 값을 지속적으로 구동 [Ch2]
Bistable element	두 안정 상태를 가진 메모리의 기본 빌딩 블록 [Ch1]
Combinational circuit	출력이 현재 입력의 조합에만 의존하는 회로 [Ch1]
Concatenation	비트를 이어붙이는 연산 {a, b}. 예: {c_out, sum} [Ch2]
Continuous assignment	assign 키워드로 net에 값을 지속 구동하는 할당 방식 [Ch2]
D Flip-Flop	master+slave D Latch 조합, rising edge에서만 값 갱신 [Ch1]
D Latch	D(data)와 CLK으로 구성, CLK=1 시 transparent [Ch1]
Decoder	N입력 2^N 출력, 입력 조합에 따라 하나의 출력만 assert [Ch1]

Dennard Scaling	트랜지스터 축소 시 전력밀도 일정 법칙 ($P \propto CV^2F$, 2000년대 종료) [Ch0]
FSM	Finite State Machine, next state logic + register + output logic [Ch1]
Gray Code	연속 값이 1비트만 차이나는 코드 (K-Map에서 사용) [Ch1]
HDL	Hardware Description Language, 하드웨어를 텍스트로 기술 [Ch2]
Implicant	하나 이상 리터럴의 AND [Ch1]
K-Map	Karnaugh Map, 불리언 수식 그래픽 간소화 도구 [Ch1]
Literal	변수 또는 그 보수 (A, $\bar{A}$) [Ch1]
Logical completeness	AND, OR, NOT만으로 모든 회로 구현 가능한 성질 [Ch1]
Maxterm	함수의 모든 변수를 포함하는 합 (OR) [Ch1]
Mealy machine	출력이 현재 상태+입력에 의존하는 FSM [Ch1]
Minterm	함수의 모든 변수를 포함하는 곱 (AND) [Ch1]
Module	Verilog의 기본 빌딩 블록, 입출력 포트를 가진 하드웨어 단위 [Ch2]
Moore machine	출력이 현재 상태에만 의존하는 FSM [Ch1]
Moore’s Law	칩 트랜지스터 수 약 18-24개월마다 2배 증가 법칙 [Ch0]
Multiplexer (Mux)	선택 신호로 여러 입력 중 하나를 출력으로 선택 [Ch1]
NAND	NOT-AND, NAND만으로 모든 논리 구현 가능 [Ch1]
Netlist	합성 결과물, 게이트와 와이어의 연결 목록 [Ch2]
POS	Product of Sums, maxterm의 AND [Ch1]
Prime implicant	다른 implicant와 더 합쳐질 수 없는 implicant [Ch1]
Procedural assignment	always/initial 블록 내에서 reg에 값 할당 [Ch2]
reg	Verilog variable 타입, procedural block에서 할당, 반드시 물리 레지스터는 아님 [Ch2]
Register	공통 CLK을 공유하는 N개 flip-flop의 뱅크 [Ch1]
RTL	Register-Transfer Level = synthesizable behavioral + dataflow [Ch2]
Sensitivity list	always 블록이 반응하는 신호 목록 [Ch2]
Sequential circuit	출력이 입력 시퀀스(현재+과거)에 의존, 메모리 있음 [Ch1]
SOP	Sum of Products, minterm의 OR [Ch1]
SR Latch	Set-Reset Latch, 교차 결합 NOR 게이트, S=R=1 금지 [Ch1]
Structural modeling	게이트/모듈 인스턴스화로 회로를 기술하는 방식 [Ch2]
Synthesis	HDL 코드를 게이트/와이어(netlist)로 변환하는 과정 [Ch2]
Testbench	설계 모듈을 검증하기 위한 시뮬레이션 코드 (합성 불가) [Ch2]
Timescale	`timescale 시간단위/정밀도, 시뮬레이션 시간 기준 설정 [Ch2]

TPU	Tensor Processing Unit, Google의 DNN 가속기 [Ch0]
wire	Verilog net 타입, 하드웨어 연결점, 값 저장 불가 [Ch2]
X (unknown)	알 수 없는 논리 값, 시뮬레이터 초기화 전 상태 [Ch1,Ch2]
Z (high-impedance)	구동되지 않는 floating 상태 [Ch1,Ch2]
Array	같은 속성 객체의 모음, 한 번에 하나의 요소만 접근 가능 [Ch3]
Arithmetic shift	<<<, >>>; 우시프트 시 부호비트로 패딩, 좌시프트는 0 패딩 [Ch3]
Bitwise operator	~, &, |, ^, ~^; 비트별 연산, z는 x로 취급 [Ch3]
Case equality	===, !==; x/z까지 정확 비교, 합성 불가 [Ch3]
Concatenation	{a, b}; 비트를 이어붙이는 연산, 피연산자 반드시 sized [Ch3]
Conditional operator	cond ? true_expr : false_expr; Mux/tri-state 모델링 [Ch3]
Continuous assignment	assign으로 net 구동, 항상 활성, 조합논리 모델링 [Ch3]
Inertial delay	게이트 RC 모델, continuous assign 기본, 짧은 펄스 무시 [Ch3]
integer	signed reg로 취급, 최소 32비트, 초기값 x [Ch3]
Logical operator	!, &&, ||; 전체 피연산자를 true/false로 평가하여 1비트 결과 [Ch3]
Logical shift	<<, >>; 빈 자리 항상 0으로 패딩 [Ch3]
Memory	ROM/RAM/register file 모델링, reg 배열로 구현 [Ch3]
Part-select (variable)	[base+:width], [base-:width]; width는 상수, base는 변수 가능 [Ch3]
Reduction operator	단항 &, |, ^; 벡터를 스칼라로 축소 [Ch3]
Replication	{n{expr}}; 반복 concatenation [Ch3]
Transport delay	와이어 지연 모델, net delay 기본, 모든 변화 전파 [Ch3]
Vector	다중 비트 신호 묶음(bus), [msb:lsb] 또는 [lsb:msb] 선언 [Ch3]
Blocking assignment	= ; 순서대로 실행, 즉시 할당, 조합논리 always 블록에서 사용 [Ch4]
case/casex/casez	다중 분기 선택문; casex: x,z don’t care, casez: z don’t care [Ch4]
forever	무한 반복 루프, $finish 또는 disable로만 종료, timing control 필수 [Ch4]
for loop	init; condition; update 형태의 반복, 합성 시 unroll됨 [Ch4]
initial block	시뮬레이션 1회 실행, 변수 초기화/테스트벤치용, ASIC 합성 불가 [Ch4]
Inter-assignment delay	#delay 문장 앞; 문장 전체 실행을 지연 [Ch4]
Intra-assignment delay	할당 RHS에 #delay; RHS 즉시 평가, 할당만 지연 [Ch4]
Named event	event 타입 선언, ->로 트리거, @로 인식; 핸드셰이킹 용도 [Ch4]

negedge	신호의 하강 에지(1→0), @(negedge clk)으로 사용 [Ch4]
Nonblocking assignment	<= ; RHS 동시 평가, 블록 끝 할당, 순차논리 always에서 사용 [Ch4]
posedge	신호의 상승 에지(0→1), @(posedge clk)으로 사용 [Ch4]
Race condition	여러 always에서 같은 신호를 blocking으로 할당 시 순서 의존 버그 [Ch4]
repeat	고정 횟수 반복 루프, counter_expr는 시작 전 1회 평가 [Ch4]
Unwanted latch	조합논리 always에서 if에 else 없거나 case에 default 없을 때 합성되는 의도치 않은 latch [Ch4]
wait	레벨 감지 이벤트 제어, 조건이 true될 때까지 대기 [Ch4]
while loop	조건이 false가 될 때까지 반복, 처음부터 false면 미실행 [Ch4]
bufif0/bufif1	tri-state 버퍼, control에 따라 활성/z 출력 [Ch5]
Dynamic hazard	출력이 3+회 변하는 현상, 경로 3+개 필요 [Ch5]
Fall delay	1/x/z → 0 전이 지연 [Ch5]
Gate primitive	Verilog 내장 14종 게이트 (and/or/xor/nand/nor/xnor/buf/not/bufif/notif) [Ch5]
Glitch	해저드에 의한 원치 않는 짧은 펄스 [Ch5]
Hazard	경로별 지연 차이로 인한 출력 불안정 현상 [Ch5]
Rise delay	0/x/z → 1 전이 지연 [Ch5]
Static hazard	출력이 일정해야 하는데 잠깐 반대 값으로 갔다가 복귀 [Ch5]
tri/tri0/tri1	tri: wire와 동일, tri0: 미구동 시 0, tri1: 미구동 시 1 [Ch5]
Turn-off delay	0/1/x → z 전이 지연 (tri-state 게이트용) [Ch5]
UDP	User-Defined Primitive, 모듈과 유사하나 다른 UDP/모듈 인스턴스화 불가 [Ch5]
wand/wor	wired net, wand: 하나가 0→0, wor: 하나가 1→1 [Ch5]

[Ch0] Introduction & Trend in Computer Systems

Dennard Scaling과 전력 문제: 전력 $P \propto C \cdot V^2 \cdot F$. 트랜지스터 축소 → C 감소, V 감소 가능 → F 증가해도 전력 유지. 하지만 2000년대 중반 이후 V를 더 낮출 수 없게 되어 Dennard Scaling 종료 → 주파수 경쟁 중단 → 멀티코어/가속기 시대로 전환.

성능 향상 원천 (AMD Lisa Su 발표): 공정기술 40%, 마이크로아키텍처 17%, 전력관리 15%, 다이크기 12%, TDP 8%, 컴파일러 8%.

특수 가속기 스펙:

칩	공정	다이	성능	전력
TPU v1	28nm	331mm²	92 TOPs (INT8)	40W
TPU v2	-	-	45 TFLOPs	-
TPU v3	-	-	105 TFLOPs	-

Tesla FSD	14nm	260mm²	73.72 TOPs	36W
AWS Inferentia	-	-	128 TOPS	-
Intel Loihi	14nm	60mm²	SNN 전용	-

[Ch1] Review: Logic Design

Logic Gates 진리표:

A	B	AND	OR	XOR	NAND	NOR	XNOR
0	0	0	0	0	1	1	1
0	1	0	1	1	1	0	0
1	0	0	1	1	1	0	0
1	1	1	1	0	0	0	1

Boolean Algebra — Axioms:

A1: B=0 if B≠1	A1': B=1 if B≠0	(Binary)
A2: $\overline{\overline{0}}=1$	A2': $\overline{\overline{1}}=0$	(NOT)
A3: 0·0=0	A3': 1+1=1	(AND/OR)
A4: 1·1=1	A4': 0+0=0	(AND/OR)
A5: 0·1=0	A5': 1+0=1	(AND/OR)

Boolean Algebra — Theorems (One Variable):

T1: B·1=B	T1': B+0=B	(Identity)
T2: B·0=0	T2': B+1=1	(Null)
T3: B·B=B	T3': B+B=B	(Idempotency)
T4: $\overline{\overline{B}}=B$		(Involution)
T5: $B\cdot\overline{B}=0$	T5': $B+\overline{B}=1$	(Complements)

Boolean Algebra — Theorems (Several Variables):

T6:	B·C = C·B	(Commutativity)
T7:	(B·C)·D = B·(C·D)	(Associativity)
T8:	B·C + B·D = B·(C+D)	(Distributivity)
T9:	B·(B+C) = B	(Covering/Absorption)
T10:	$B\cdot C + B\cdot\overline{C} = B$	(Combining) ★★★
T11:	$BC + \overline{B}D + CD = BC + \overline{B}D$	(Consensus)
T12:	$(B_0\cdot B_1...)^{\overline{}} = \overline{B}_0+\overline{B}_1...$	(De Morgan's) ★★★

K-Map 규칙: 1. 1을 모두 덮는 최소 수의 직사각형 2. 각 직사각형은 1만 포함 3. 크기는 2의 거듭제곱 (1,2,4,8...) 4. 가능한 한 크게 5. 가장자리 wrap-around 가능 6. 하나의 1이 여러 원에 포함 가능

K-Map 간소화 예제:

$$
\begin{aligned}
F &= \overline{A}\overline{B}C + \overline{A}BC + A\overline{B}C \\
&= \overline{A}C(\overline{B}+B) + A\overline{B}C \qquad \text{(T10 적용)} \\
&= \overline{A}C + A\overline{B}C \\
&= C(\overline{A} + A\overline{B}) \\
&= C(\overline{A}+A)(\overline{A}+\overline{B}) \qquad \text{(T8' 분배)} \\
&= C(\overline{A}+\overline{B})
\end{aligned}
$$

SR Latch:

```
R──→┤NOR├──o──→ Q
    │ N1│    \
    └───┘     \
        \      |
```

S	R	Q	$\overline{Q}$
0	0	Qprev	$\overline{Q}$prev
0	1	0	1

```
S──→┤NOR├──o──→ Q̄          1  0 | 1      0
    | N2|                   1  1 | 0      0 (금지!)
    └───┘
```

D Flip-Flop (Master-Slave):

```
            CLK           CLK
D ──→┤D  Q├─N1─→┤D  Q├──→ Q
     |L1 Q̄|     |L2 Q̄|──→ Q̄
     └────┘     └────┘

     master       slave

CLK=0: master transparent, slave opaque (D→N1)
CLK=1: master opaque, slave transparent (N1→Q)
→ Rising edge에서만 D가 Q로 복사됨
```

FSM 설계 절차: 1. State Transition Diagram 작성 2. State Transition Table 작성 3. State Encoding (이진 코드 부여) 4. Boolean Equations 유도 (Next State + Output) 5. Schematic 작성

FSM 예제 — Traffic Light Controller:

```
State|Encoding| LA    | LB
─────┼────────┼───────┼──────
 S0  |  00    | green | red
 S1  |  01    | yellow| red
 S2  |  10    | red   | green
 S3  |  11    | red   | yellow

Next State:
  S'₁ = S₁ ⊕ S₀
  S'₀ = S̄₁·S̄₀·T̄_A + S₁·S̄₀·T̄_B

Output:
  L_A1 = S₁,  L_A0 = S̄₁·S₀
  L_B1 = S̄₁,  L_B0 = S₁·S₀

→ Moore Machine (출력이 상태에만 의존)
```

[Ch2] Introduction to Verilog

Module 기본 구조:

```
module module_name (input  a,
                    input  b,
                    output y);
  // 회로 기술
endmodule
```

- ANSI-C 스타일 포트 선언 사용 (Verilog-2001)
- module 선언 끝에 세미콜론(;) 필수, endmodule 뒤에는 없음

wire vs reg 사용 규칙:

```
wire: assign문의 LHS, module의 input/inout port
reg:  always/initial 블록의 LHS, output port 가능 (output reg)

주의: reg ≠ 물리적 레지스터! 합성 시 조합논리가 될 수도 있음
```

세 가지 Assignment:

```
(1) Continuous: assign temp = a | b;     → wire에 사용, 조합논리
(2) Procedural: always @(*) y = a & b;   → reg에 사용, always 블록 내
(3) Procedural Continuous: assign/deassign → 특수 용도, 잘 안 씀
```

세 가지 모델링 레벨 코드 비교 (Full Adder):

```
// (1) Structural — 게이트 인스턴스화
module full_adder_structural(input x, y, c_in,
                             output sum, c_out);
  wire s1, c1, c2;
  half_adder ha_1(x, y, s1, c1);
  half_adder ha_2(c_in, s1, sum, c2);
  or(c_out, c1, c2);
endmodule

// (2) Dataflow — assign + 연산자
module full_adder_dataflow(input x, y, c_in,
                           output sum, c_out);
  assign {c_out, sum} = x + y + c_in;
endmodule

// (3) Behavioral — always 블록
module full_adder_behavioral(input x, y, c_in,
                             output reg sum, c_out);
  always @(x, y, c_in)
    {c_out, sum} = x + y + c_in;
endmodule
```

합성 회로도 (세 가지 모두 동일한 하드웨어로 합성):

```
Full Adder 합성 결과:

x ──→┤ XOR ├─→ s1 ──→┤ XOR ├─→ sum
y ──→┤     │         │     │
     └─────┘    c_in──→┤     │
                       └─────┘

x ──→┤ AND ├─→ c1 ──→┤     │
y ──→┤     │         │ OR  ├─→ c_out
     └─────┘         │     │
c_in─→┤ AND ├─→ c2 ──→┤     │
s1 ──→┤     │         └─────┘
      └─────┘

또는 간단히:
x ──→┤ Half   ├─s1──→┤ Half   ├──→ sum
y ──→┤ Adder  │      │ Adder  │
     │ ha_1   ├─c1─┐ │ ha_2   ├─c2─┐
     └────────┘    │ └────────┘    │
            c_in───┘               │
                  └──→┤ OR ├──→ c_out ←─┘
```

RTL = synthesizable behavioral + dataflow (업계 표준)

숫자 표현 형식:

```
형식: <size>'<base>value
예: 16'b0101_1100_0000_1101
    8'hAA  →  10101010
    'hF    →  32비트 0...01111

크기<값: MSB truncation (4'b11110010 → 0010)
```

```
크기>값: MSB padding (0으로, 또는 x/z로)
음수: -4'b1001 → 2의 보수
_: 가독성만, ?: z와 동일
```

Port Connection:

```
// Named association (추천!)
half_adder ha_2(.x(cin), .y(s1), .s(s), .c(c2));

// Positional association (비추천)
half_adder ha_1(x, y, s1, c1);
```

연산자 우선순위 (높→낮):

```
~  >  *,/,%  >  +,-  >  <<,>>  >  <<<,>>>
>  <,<=,>,>=  >  ==,!=  >  &,~&  >  ^,~^
>  |,~|  >  ?:
```

Simulation 핵심:

```
`timescale 1ns / 100ps      // 시간단위 / 정밀도
$display("...", vars);       // 한 번 출력
$monitor("...", vars);       // 값 변화시마다 자동 출력
$finish;                     // 시뮬레이션 종료
always #5 clk = ~clk;       // 주기 10ns 클럭 생성
```

[Ch3] Dataflow Modeling

Continuous Assignment 규칙: - LHS는 반드시 net(wire)의 scalar/vector/concatenation. 절대 reg 아님! - RHS에는 reg, function call 등 사용 가능 - 항상 활성(Always Active): RHS 변하면 즉시 LHS 갱신 - net은 한 번만 선언, 여러 assign은 가능하지만 비추천

```
wire out;
assign out = in1 & in2;     // regular (추천)
wire out = in1 & in2;       // implicit (OK)
assign out = in1 & in2;     // implicit net decl (비추천)
```

Delay 모델:

```
Inertial delay: assign #10 out = in1 & in2;
  → 짧은 펄스(< delay) 무시, 게이트 RC 모델
Transport delay: wire #10 out; assign out = in1 & in2;
  → 모든 변화 전파, 와이어 지연 모델
```

Signed/Unsigned 상수 — 시험 함정:

```
4'sb1001  → 비트 1001 → signed → -7
5'sb1001  → 비트 01001 → signed → +9
4'shf     → 비트 1111 → signed → -1
4'hf      → 비트 1111 → unsigned → 15
4'sd12    → 비트 1100 → signed → -4 (12=1100)
5'sd12    → 비트 01100 → signed → +12
-4'sb0010 → -(2) → 비트 1110

원칙: 's'는 비트패턴 안 바꿈, 해석만 바꿈
```

Signed/Unsigned 비교 함정:

```
wire signed [3:0] a = -4'd4;   // 1100, signed→-4
wire unsigned [3:0] b = 4'd5;  // 0101, unsigned→5
```

```
(a > b) → 1 (true!)  // b가 unsigned → 모두 unsigned로 비교
                       // 1100=12 > 0101=5 → true
```

Equality (== vs ===):

```
4'b1x0z == 4'b1x0z   → x  (x/z 포함 → 결과 x)
4'b1x0z === 4'b1x0z  → 1  (x/z까지 정확 비교, 합성불가)
```

등호 비교 시 sign extension:

```
wire signed [5:0] a = -6'd4;  // 111100
wire signed [3:0] b = -4'd4;  // 1100 → sign ext → 111100
(a == b) → 1
```

연산자 총정리 코드 예제:

```
// 4-bit Adder
assign {c_out, sum} = x + y + c_in;

// 4-bit Adder/Subtractor
assign t = y ^ {4{c_in}};           // c_in=1→~y
assign {c_out, sum} = x + t + c_in; // c_in=1→x-y

// Parity Generator (reduction XOR)
assign ep = ^x;         // even parity
assign op = ~ep;        // odd parity

// All-Zero / All-One Detector
assign zero = ~(|x);    // OR reduction → NOT
assign one  = &x;       // AND reduction

// 4:1 Mux (conditional 중첩)
assign out = s1 ? (s0 ? i3 : i2) : (s0 ? i1 : i0);
```

합성 회로도:

```
[4-bit Adder]
x[3:0] ──→┤        ├──→ sum[3:0]   ({c_out, sum} = 5비트)
y[3:0] ──→┤  ADD   │
c_in   ──→┤        ├──→ c_out      (MSB = carry)
          └────────┘


[4-bit Adder/Subtractor]
y[3:0]──→┤ XOR ├──→ t[3:0]──→┤        ├──→ sum[3:0]
c_in ──→┤{4{}}├              │  ADD   │
         └─────┘    x[3:0]──→┤        ├──→ c_out
                    c_in ──→┤        │
                            └────────┘

c_in=0: t=y     → x+y    (덧셈)
c_in=1: t=~y    → x-y    (뺄셈, 2의 보수)

[Parity Generator — 9-bit]

x[0]──→┤XOR├──→┤XOR├──→┤XOR├──→┤XOR├──→┤XOR├──→┤XOR├──→┤XOR├──→┤XOR├
 ep
  x[1]──→┤   │   │   │   │   │   │   │   │   │   │   │   │   │   │
|
         └───┘   └───┘   └───┘   └───┘   └───┘   └───┘   └───┘
└───┘
  x[2]──→──┘  x[3]─┘  x[4]─┘  x[5]─┘  x[6]─┘  x[7]─┘  x[8]─┘
  ep──→┤NOT├──→ op
```

```
[All-Zero Detector]
x[0]──→┤OR├──→┤OR├──→...──→┤OR├──→┤NOT├──→ zero
x[1]──→┤  │   │  │         │  │
       ...    ...           │  │
x[7]──────────────────────→┤  │

[All-One Detector]
x[0]──→┤AND├──→┤AND├──→...──→┤AND├──→ one
x[1]──→┤   │   │   │         │   │
       ...     ...            │   │
x[7]──────────────────────────→┤   │

[4:1 Mux — conditional 중첩]
i0──→┤MUX├          ┌──→┤MUX├──→ out
i1──→┤2:1├──→──────┘   │2:1│
     └───┘    s0        │   │
i2──→┤MUX├──→──────────┤   │
i3──→┤2:1│       s1──→┤   │
     └───┘            └───┘
     s0
```

Shift 연산:

```
Logical >>1:  10101010 → 01010101 (0 패딩)
Arith   >>>1: 10101010 → 11010101 (부호비트 1 패딩, -86→-43)
Logical <<1:  10101010 → 01010100 (0 패딩)
Arith   <<<1: 10101010 → 01010100 (0 패딩, 부호 비보존!)
```

Bitwise x/z 진리표:

```
&  | 0  1  x        |  | 0  1  x
───┼──────────       ──┼──────────
0  | 0  0  0        0  | 0  1  x
1  | 0  1  x        1  | 1  1  1
x  | 0  x  x        x  | x  1  x
(z는 x로 취급)
```

Variable Data Types 초기값:

```
reg, integer, time → 초기값 x
real, realtime     → 초기값 0.0
```

Vector endianness:

```
reg [7:0] a = 14;   // a[3:0]=1110 ✓
reg [0:7] b = 14;   // b[0:3]=0000, b[3:0]은 불법!
```

Variable part-select:

```
wire [15:0] v;
v[8+:8]  // = v[15:8]  (base=8, 위로 8비트)
v[7-:8]  // = v[7:0]   (base=7, 아래로 8비트)
// width는 상수, base는 변수 가능
```

[Ch4] Behavioral Modeling

Blocking vs Nonblocking — 5가지 가이드라인 (필수 암기!):

```
1. 순차 논리: always @(posedge clk) + nonblocking (<=)
```

```
2. 간단한 조합 논리: assign (continuous assignment)
3. 복잡한 조합 논리: always @(*) + blocking (=)
4. 같은 신호를 여러 always/assign에서 할당하지 마라
5. 같은 always 블록에서 blocking과 nonblocking을 섞지 마라
```

Blocking vs Nonblocking 동작 차이:

```
// Blocking: 순서대로, 즉시 할당
initial begin
  x = #5 1'b0;   // time 5
  y = #3 1'b1;   // time 8 (5+3)
  z = #6 1'b0;   // time 14 (8+6)
end

// Nonblocking: RHS 동시 평가, 블록 끝 할당
initial begin
  x <= #5 1'b0;  // time 5
  y <= #3 1'b1;  // time 3 (동시 시작!)
  z <= #6 1'b0;  // time 6
end
```

Shift Register — 핵심 예제:

```
// Blocking (잘못! FF 1개만 합성)
always @(posedge clk) begin
  qout[0] = sin;       // 즉시 변경
  qout[1] = qout[0];   // sin의 새 값 사용!
  qout[2] = qout[1];   // 역시 sin!
  qout[3] = qout[2];   // 모두 sin! → FF 1개
end

// Nonblocking (올바름! FF 4개 체인)
always @(posedge clk) begin
  qout[0] <= sin;       // 모든 RHS를 이전 값으로 평가
  qout[1] <= qout[0];   // qout[0]의 이전 값
  qout[2] <= qout[1];
  qout[3] <= qout[2];   // → FF 4개 체인!
end
// 더 좋은 표현: qout <= {qout[2:0], sin};
```

합성 회로도 비교:

```
[Blocking — 잘못된 합성 (FF 1개)]
sin ──→┌ DFF ┐──→ qout[0] ═══ qout[1] ═══ qout[2] ═══ qout[3]
       └─────┘                (단순 wire 연결, FF 없음!)
        clk
→ 모든 비트가 동일 값. shift register가 아님!

[Nonblocking — 올바른 합성 (FF 4개 체인)]

sin──→┌DFF┐──→qout[0]──→┌DFF┐──→qout[1]──→┌DFF┐──→qout[2]──→┌DFF┐──→(

      └───┘              └───┘              └───┘

└───┘

                                  clk (공유)
→ 각 비트가 이전 비트의 이전 값을 받음. 올바른 shift register!
```

Race Condition 방지:

```
// Blocking → race! (순서 의존)
always @(posedge clock) x = y;
```

```
always @(posedge clock) y = x;

// Nonblocking → 올바른 swap
always @(posedge clock) x <= y;
always @(posedge clock) y <= x;
```

Unwanted Latch 방지:

```
// 나쁜 예: else 없음 → latch!
always @* begin
  if (cond1) A = val1;
  else if (cond2) A = val2;
  // cond1, cond2 모두 아니면? → A 유지 → latch!
end

// 올바른 예: else로 모든 경우 커버
always @* begin
  if (cond1) A = val1;
  else if (cond2) A = val2;
  else A = default_val;   // → latch 방지
end
```

D FF / D Latch / Combinational 비교:

```
// D Flip-Flop (순차)
always @(posedge CLK) Q <= D;

// D Latch (순차)
always @(CLK, D) if (CLK) Q <= D;

// Combinational (조합)
always @(*) y = ~a;   // 합성: 메모리 소자 없음!
```

합성 회로도 비교:

```
[D Flip-Flop — @(posedge CLK)]
D[3:0]──→┤ D   Q ├──→ Q[3:0]
         │  DFF  │
CLK ──→──┤>      │
         └───────┘
→ 4개의 DFF (4-bit register)

[D Latch — @(CLK, D) if(CLK)]
D[3:0]──→┤ D   Q ├──→ Q[3:0]
         │ LATCH │
CLK ──→──┤ EN    │
         └───────┘
→ 4개의 D Latch (CLK=1일 때 transparent)

[Combinational — @(*)]
a[3:0]──→┤ NOT ├──→ y[3:0]
         └─────┘
→ 인버터만! 메모리 소자 없음 (reg 타입이지만 FF/Latch 아님)
```

Counter with 비동기 리셋:

```
always @(negedge clock or posedge clear) begin
  if (clear) qout <= 4'd0;       // 비동기 리셋 우선
  else       qout <= qout + 1;   // negedge에서 카운트
end
```

합성 회로도:

```
                    ┌─────┐
qout[3:0]──→──→──┤  +  ├──→┤ 0 │
           1──→──┤     │   │MUX├──→┤ D   Q ├──→ qout[3:0]
                    └─────┘   │   │   │ DFF   │
            4'd0 ──────────→┤ 1 │   │  R    │
                              └───┘   └───────┘
                  clear──→──┤sel│     │     │
                              └───┘   neg   posedge
                                      clk   clear
→ Adder + Mux + DFF with async reset
→ clear=1이면 즉시 리셋 (클럭 무관, 비동기)
```

case 4:1 Mux 합성 회로도:

```
        case (S)
          2'b00: Y = I0;
          2'b01: Y = I1;
          2'b10: Y = I2;
          2'b11: Y = I3;
        endcase

I0──→┤       ├
I1──→┤  4:1  ├──→ Y
I2──→┤  MUX  │
I3──→┤       │
     └───────┘
S[1:0]──→┤sel│
```

casex (data) // x,z를 don't care로 4'bxxx1: out = 0; // LSB=1 → trailing zero 0개 4'bxx10: out = 1; 4'bx100: out = 2; default: out = 3'b111; endcase

```
**Timing Control:**
```

Inter-assignment: #10 a = b; → 10단위 후 문장 전체 실행 Intra-assignment: a = #10 b; → RHS 즉시 평가, 10단위 후 할당 a <= #10 b; → RHS 즉시 평가, 10단위 후 할당 (다른 문장 차단 안 함)

```
**Loop:**

```verilog
// for — 합성 시 unroll
for (i=0; i<=7; i=i+1)
  if (data[i]==0) out = out + 1;

// repeat — 고정 횟수
repeat (32) begin state[i]=0; i=i+1; end

// forever — 클럭 생성
forever begin #10 clock<=1; #5 clock<=0; end
```

[Ch5] Structural Modeling

14 Gate Primitives:

```
and/or/xor/nand/nor/xnor: 다입력 단출력
  gate_name [inst] (output, in1, in2, ..., inN);
```
```

```
buf/not: 단입력 다출력
  gate_name [inst] (out1, out2, ..., outN, input);

bufif0/bufif1/notif0/notif1: tri-state (출력, 입력, control)
  gate_name [inst] (output, input, control);
```

Net Type Resolution (다중 드라이버):

```
  wire/tri │ 0  1  x  z     wand │ 0  1  x  z     wor │ 0  1  x  z
  ─────────┼──────────────  ─────┼──────────────
─────┼──────────────

     0     │ 0  x  x  0       0  │ 0  0  0  0       0 │ 0  1  x  0
     1     │ x  1  x  1       1  │ 0  1  x  1       1 │ 1  1  1  1
     x     │ x  x  x  x       x  │ 0  x  x  x       x │ x  1  x  x
     z     │ 0  1  x  z       z  │ 0  1  x  z       z │ 0  1  x  z

  tri0: z+z→0 (pull-down)    tri1: z+z→1 (pull-up)
```

Gate Delay 지정:

```
        and #(5) a1(b,x,y);                    // 단일 (모든 전이 동일)
        and #(t_rise, t_fall) a2(c,a,z);       // rise/fall
        or  #(t_rise, t_fall, t_off) o1(f,b,c); // rise/fall/turn-off
        // min:typ:max 형식 가능: #(10:12:15, 12:15:20)
        // 생략: 값없음→0, 1개→모두동일, 2개→t_off=min(rise,fall)
```

Delay 계산 테이블 (발췌):

```
  0→1: rise delay    1→0: fall delay    x→z: turn-off delay
  0→x: min(rise,fall)  (3개 지정 시: min(r,f,off))
  0→z: turn-off      (2개 지정 시: min(rise,fall))
```

Hazard:

```
  Static-1 hazard: 출력 1이어야 하는데 잠깐 0 glitch
  Static-0 hazard: 출력 0이어야 하는데 잠깐 1 glitch
  Dynamic hazard: 3+회 변동, 경로 3+개 필요

  K-Map에서 발견: 인접한 1이 같은 루프에 없으면 hazard
  제거: 중복 항 추가 (redundant term)

  예: f = xy + x̄z → hazard
      f = xy + x̄z + yz → hazard 제거 (yz가 전이 구간 커버)
```

Static Hazard 예제 회로:

```
  x──→┤AND├──→a──→┤     │
  y──→┤#5 │        │ OR ├──→f   f = xy + x̄z
      └───┘        │#5  │
  x──→┤NOT├──→c─┐  │    │   y=1,z=1에서 x: 1→0
      │#5 │     │  └────┘   a 경로: 5ns, b 경로: 10ns
      └───┘     │           → a가 먼저 0 → f 잠깐 0 (glitch!)
  z──────────┤AND├──→b──→┘
             │#5 │
      c──→───┤   │
             └───┘
```